

برنامه نویسی پویا (Dynamic programming)

- یک روش حل مسائل بزرگ و سخت، تقسیم کردن آن به تعدادی نمونه مسئله کوچکتر و سپس حل جواب آنهاست.
- در برخی مسائل، زیرمسئله ها مستقل از هم نیستند یعنی زیرمسئله های کوچکتر وابستگی و همپوشانی دارند.
- در اینگونه مسائل، روش تقسیم و غلبه ناکارآمد است. زیرا زیرمسئله ها کوچکتر که وابسته هستند به صورت تکراری محاسبه می گردند (مثلاً الگوریتم بازگشتی سری فیبوناچی)
- برنامه نویسی پویا همانند تقسیم و غلبه، مسئله را با ترکیب جواب های به دست آمده برای زیرمسئله ها حل می کند.

روش تقسیم و حل برای الگوریتم سری فیبوناچی

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ... **سری فیبوناچی**

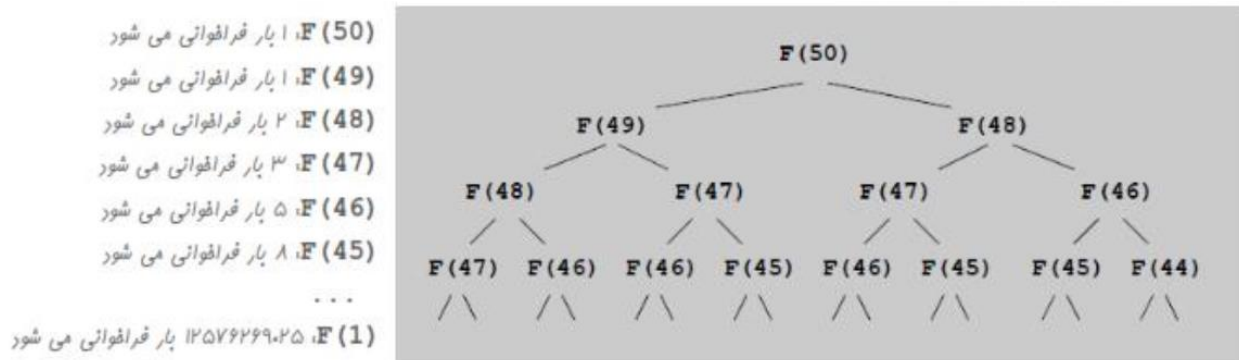
حل بازگشتی سری فیبوناچی

```
public static long F(int n)
{
    if (n == 0) return 0;
    if (n == 1) return 1;
    return F(n - 1) + F(n - 2);
}
```

پیچیدگی زمانی

$$F(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n \geq 2 \end{cases}$$

مثال: آیا راه حل بازگشتی برای محاسبه جمله شماره پنجاه سری فیبوناچی $F(50)$ مناسب است؟



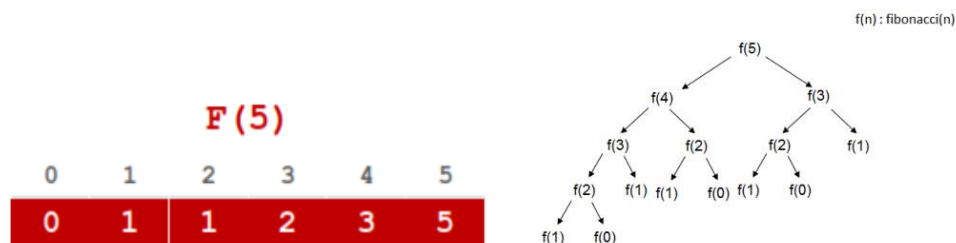
برای حل این مشکل ما می توانیم مقادیر تکراری (زیر مسئله های مشترک) را حل کرده و پاسخ آنها را ذخیره نموده و در صورت نیاز به حل مجدد زیرمسئله های مشترک از پاسخ ذخیره شده استفاده کنیم.

```
public static long F(int n)
{
    if (n == 0) return 0;
    long[] F = new long[n + 1];
    F[0] = 0;
    F[1] = 1;
    for (int i = 2; i <= n; i++)
        F[i] = F[i - 1] + F[i - 2];
    return F[n];
}
```

این کد تنها با ۴۹ عمل جمع می تواند جمله شماره پنجاه را محاسبه کند

برنامه نویسی پویا، از این لحاظ که نمونه به نمونه های کوچکتر تقسیم می شود، مشابه روش تقسیم و حل است ولی در این روش، نخست نمونه های کوچک تر حل می شود، نتایج ذخیره می گردد و هر گاه به یکی از آن ها نیاز پیدا شد، به جای محاسبه دوباره کافی است بازیابی گردد

برای مثال جمله شماره ۵



الگوریتم های تقسیم و غلبه مساله را به زیر مسائل تقسیم می کند و زیر مسائل را به صورت بازگشتی حل می کنند و سپس جواب زیر مساله ها را با هم ترکیب می کند تا جواب مساله کلی به دست آید. در مقابل برنامه ریزی پویا زمانی کاربرد دارد که زیر مساله ها با هم همپوشانی دارد یعنی وقتی زیرمساله ها خود زیر مسائل مشترک دارند.

روش تقسیم و حل به دلیل حل مکرر زیرمسئله های مشترک در برخی از مسائل مانند سری فیبوناچی ناکارآمد است. اما روش برنامه نویسی پویا با یک رویکرد پایین به بالا، ابتدا زیرنمونه های کوچکتر را حل کرده و جواب آنها را ذخیره می کند.

برنامه نویسی پویا را می توان به ۴ مرحله تقسیم کرد:

- توصیف ساختار یک جواب بهینه
- تعیین مقدار جواب بهینه به صورت بازگشتی
- محاسبه مقدار یک جواب بهینه به سبک پایین به بالا
- ساختن یک جواب بهینه از اطلاعات محاسبه شده

مسئله برش بهینه یک میله

فرض کنید می خواهیم یک میله فولادی به طول n را برش بزنیم. مشتری های مختلف برای خرید قطعات کوچک با اندازه های متفاوتی از این میله وجود دارند. اما قیمت قطعات کوچک تر دقیقاً بر اساس اندازه آن ها تعیین نمی شود.

در حالت کلی طول تمام قطعات یک عدد صحیح بر حسب متر و قیمت قطعات مختلف در یک آرایه $p[1..n]$ موجود است که $p[i]$ نشان دهنده ارزش یک قطعه با طول i می باشد هدف برش میله به گونه ای است که در نهایت بیشترین سود را داشته باشد

ورودی: یک طول به طول n و جدول قیمت های p_i برای $i = 1, 2, \dots, n$.

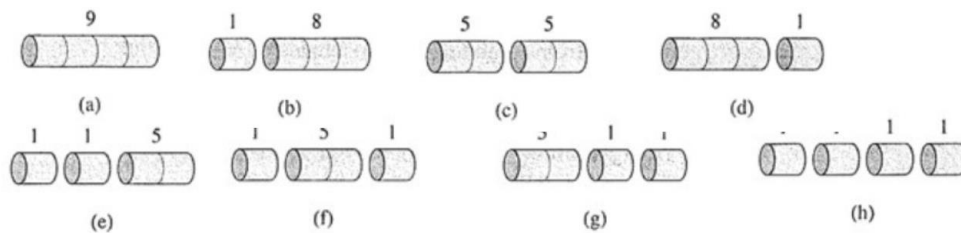
خروجی: بیشترین درآمد قابل دستیابی برای میله هایی که مجموع طول های آنها n است، محاسبه می شود.

اگر p_n به اندازه کافی بزرگ باشد، یک راه حل بهینه ممکن است نیاز به هیچ برشی نداشته باشد، یعنی میله را به طول n اینچ باقی بگذارید.

مثال: به طور مثال جدول زیر را در نظر بگیرید

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

اگر میله ای به طول ۴ داشته باشیم، به ۸ روش مختلف می توان برش زد:



می توانیم یک میله با طول n را به 2^{n-1} روش مختلف برش دهیم، چرا که برای هر نقطه ای در فاصله از انتهای چپ، برای $1, 2, \dots, n-1$ یک انتخاب مستقل مبنی بر برش یا عدم برش در آن نقطه داریم. بنابراین یک الگوریتم نمایی است

بهترین روش این است که آن را به دو قطعه ۲ اینچی برش دهیم و درآمد $p_2 + p_2 = 5 + 5 = 10$

اگر اولین برش در i انجام گیرد (یعنی اولین قطعه به طول i باشد) آنگاه سود بهینه برابر است با p_i به اضافه سود بهینه حاصل از برش یک قطعه بزرگ به اندازه $n-i$ می باشد. بنابراین اگر حداکثر سود حاصل از برش یک چوب به اندازه n را با r_n نمایش دهیم آنگاه می توانیم بنویسیم.

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$

به عبارت ساده تر ما باید مقدار r_i را برای i های مختلف بدست آورده و با استفاده از آن مقدار بهینه کلی را پیدا کنیم. ما جدول زیر را برای مثال بدست می آوریم.

i	r_i	optimal solution
1	1	1 (no cuts)
2	5	2 (no cuts)
3	8	3 (no cuts)
4	10	2 + 2
5	13	2 + 3
6	17	6 (no cuts)
7	18	1 + 6 or 2 + 2 + 3
8	22	2 + 6

حال باید از بین تمام حالت های مختلف، حالت بهینه (در این جا بیشترین سود) را پیدا کنیم. لیستی از تمام حالت های ممکن در زیر آورده شده است و ما از بین آنها، بهینه ترین را انتخاب خواهیم کرد

$$\begin{aligned}
 & p_n: \text{درآمد حاصل از عدم برش,} \\
 & r_1 + r_{n-1}: \text{بیشترین درآمد از یک میله ۱ اینچی و یک میله } n-1 \text{ اینچی,} \\
 & r_2 + r_{n-2}: \text{بیشترین درآمد از یک میله ۲ اینچی و یک میله } n-2 \text{ اینچی, } \dots \\
 & \quad \cdot \\
 & \quad \cdot \\
 & \quad \cdot \\
 & r_{n-1} + r_1
 \end{aligned}$$

یعنی به عبارت دیگر:

$$r_n = \max \{p_n, r_1 + r_{n-1}, r_2 + r_{n-2}, \dots, r_{n-1} + r_1\}$$

یافتن زیرمسئله های بهینه: برای حل مسئله اصلی با اندازه n ، زیرمسائل با اندازه های کوچکتر را حل کنید. پس از انجام یک برش، دو زیرمساله باقی می ماند. راه حل بهینه برای مسئله اصلی شامل راه حل های بهینه برای زیرمسائل است. ممکن است زیرمسائل را مستقل از هم حل کنید.

مثال: برای $n = 7$ ، یکی از راه‌حل‌های بهینه ایجاد برش در ۳ اینچ است. این برش مسئله اصلی را به دو زیر مسئله با طول‌های ۳ و ۴ ایجاد می‌کند. باید هر دو را بهینه حل کنید.

راه‌حل بهینه برای مسئله طول ۴، برش به ۲ قطعه، هر کدام به طول ۲، که از آن در راه‌حل بهینه مسئله اصلی با طول ۷ استفاده می‌شود.

یک روش ساده برای شکستن مسئله به زیرمسئله

هر راه‌حل بهینه حتما دارای یک سمت چپ‌ترین برش است. به عبارت دیگر، برشی وجود دارد که یک قطعه اول به طول i از انتهای چپ جدا می‌کند و یک قطعه باقیمانده به طول $n - i$ در سمت راست باقی می‌گذارد.

- می‌توان گفت ما موفق شدیم اولین نقطه برش که سمت چپ‌ترین هم است، را پیدا کنیم. حالا کافی است قسمت باقیمانده (که طول آن $n-i$ است) را بصورت بهینه تقسیم بندی کنیم.
- پس ما به جای دو زیرمسئله کافی است، یک زیرمسئله را حل کنیم. ما باید زیرمسئله قسمت باقیمانده را حل کنیم.
- اگر بخواهیم مسئله را با این روش به زیرمسئله تبدیل کنیم، آن گاه ممکن است با یک مشکل روبرو شویم.
- مشکل: ممکن است، راه حل بهینه‌ای هم وجود داشته باشد که در آن نیاز به انجام هیچ برشی نباشد. در این صورت مقدار i را باید چند قرار دهیم؟ در این صورت کافی است مقدار $i=n$ قرار دهیم و طول باقیمانده را صفر در نظر بگیریم. سود ما برای طول صفر قطعا صفر خواهد بود. ($r_0 = 0$)
- با توجه با این شیوه جدید تقسیم کردن، نسخه ساده‌تری از معادله برای r_n بصورت زیر خواهد بود:

$$r_n = \max\{p_i + r_{n-i} : 1 \leq i \leq n\}$$

راه‌حل بازگشتی بالا به پایین

ما معادله ساده‌تر را که برای r_n بدست آوردیم به تابع $\text{CUT-ROD}(p, n)$ پیاده سازی می‌کنیم. با فراخوانی این تابع مقدار بازگشتی سود بهینه ما r_n خواهد بود.

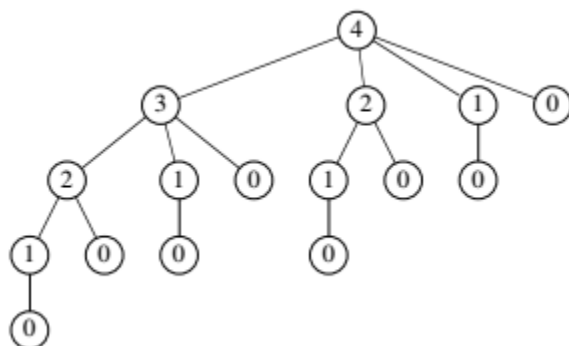
```

CUT-ROD( $p, n$ )
  if  $n == 0$ 
    return 0
   $q = -\infty$ 
  for  $i = 1$  to  $n$ 
     $q = \max\{q, p[i] + \text{CUT-ROD}(p, n - i)\}$ 
  return  $q$ 

```

این تابع پاسخ را برای ما پیدا می‌کند، اما بسیار ناکارآمد است. اگر آن را کدنویسی و اجرا کنید، ممکن است برای $n = 40$ بیش از یک ساعت طول بکشد. تقریبا با هر یک واحد افزایش n زمان اجرا دو برابر می‌شود.

چرا اینقدر ناکارآمد؟ CUT-ROD به طور مکرر خود را فراخوانی می‌کند، حتی روی زیرمسائلی که قبلاً حل کرده است. در اینجا یک درخت از فراخوانی‌های بازگشتی برای $n = 4$ آورده شده است. داخل هر گره مقدار n برای فراخوانی نشان داده شده است:



درخت فراخوانی بازگشتی برای برش میله دارای تعداد زیادی زیرمساله تکراری است. زیرمساله برای اندازه ۲ را دو بار، برای اندازه ۱ را چهار بار و برای اندازه ۰ را هشت بار حل می‌کند.